



GOVERNMENT POLYTECHNIC MUZAFFARPUR

(Under Department of Science, Technology &
Technical Education Bihar)



LAB MANUAL

DataBase Management System Lab

Course Code: 2018402

INDEX

Sr. No.	Title of Practical	CO
1	Know installation of Oracle / MySQL / MongoDB	CO1
2	Exercise on creating tables, inserting records, and updating records	CO2
3	Exercise on modifying the structure of the table	CO2
4	Exercise on SELECT command	CO3
5	Exercise on querying the table using clauses: WHERE, ORDER BY, IN, AND, OR, NOT	CO3
6	Exercise on various group functions	CO4
7	Exercise on number, character, conversion, and date functions	CO4
8	Exercise on set operators	CO3
9	Exercise on subqueries	CO3
10	Exercise on joins	CO3
11	Exercise on various date and number format models	CO4
12	Exercise on creating tables with integrity constraints	CO2
13	Exercise on creation and dropping of database and collections in MongoDB	CO5
14	Exercise on MongoDB CRUD commands and sorting of documents	CO5

COURSE INFORMATION

Section A: Course Outcomes (COs)

CO No.	Course Outcome Statement
CO1	Install and configure relational (Oracle/MySQL) and NoSQL (MongoDB) database systems.
CO2	Create, modify, and manage database structures, including tables, collections, and integrity constraints.
CO3	Insert, update, delete, and retrieve data using SQL queries, clauses, joins, subqueries, and set operators.
CO4	Apply SQL functions, grouping, formatting, and aggregation techniques to analyze database records.
CO5	Perform MongoDB operations such as CRUD on documents and manage databases and collections.

Programme Outcomes (POs)

PO	Programme Outcome
PO-1	Basic and Discipline Specific Knowledge
PO-2	Problem Analysis
PO-3	Design / Development of Solutions
PO-4	Engineering Tools
PO-5	Engineering Practices for Society, Sustainability and Environment
PO-6	Project Management
PO-7	Life Long Learning

Programme-Specific Outcomes (PSOs)

PSO	Programme-Specific Outcome
PSO1	Use basic computer science concepts like programming, data structures, database management, and computer networks to create and keep up-to-date simple software and computer solutions that can be used in real life.
PSO2	Use modern computing tools, programming languages and software development practices to design, implement and test applications, demonstrating problem-solving ability and readiness for entry-level technical roles or higher education.

Section B: Course Articulation Matrix (CO-PO Mapping)

CO / PO / PSO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO1	3	2	1	3	–	1	2	2	2

CO / PO / PSO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO2	3	3	3	3	–	1	2	2	3
CO3	3	3	3	3	–	1	2	3	3
CO4	3	3	2	3	–	1	2	2	3
CO5	3	2	2	3	–	1	2	2	3

Legend: 3 = High Correlation | 2 = Medium Correlation | 1 = Low Correlation | – = No Mapping

Section C: General Lab Instructions

- Read the practical thoroughly before coming to the lab.
- Ensure all software tools (MySQL / Oracle / MongoDB) are installed and running before starting.
- Save all SQL scripts and output files with proper naming (e.g., Prac1_1_Installation.sql).
- Write the aim, SQL code, output, and conclusion in the lab journal after each practical.
- Do not copy code — understand the logic and implement it independently.
- Viva-voce questions will be asked at the end of each practical session.
- Maintain the lab journal up to date and submit it on time.

Assessment

S. No.	Category	Criteria of Assessment	Marks
1	Database Design & Implementation	Database and table creation with appropriate data types	2
2		Appropriate use of keys and constraints	2
3		Correct formulation and execution of SQL queries	2
4	Result Analysis & Documentation	Accuracy of query execution and output	2
5		Correct use of SQL commands and DBMS concepts	2
6		Completeness and proper maintenance of lab record	2
7	Viva-Voce	Conceptual understanding of DBMS and SQL	3
		Total	15

Rubrics for Evaluation

S.No	Name of Student	Database Design & Implementation(6)	Result Analysis & Documentation(6)	Viva (3)	Total (15)
1					

Practical No: 1.1

1. Aim / Practical Statement

Know installation of Oracle / MySQL / MongoDB

2. Practical Significance

Installing and configuring database management systems is the first critical step in any database-driven project. This practical ensures that students can set up the database environment independently, which is essential for all subsequent database activities in the lab and in industry.

3. Relevant Theory

Database Management System (DBMS)

- A DBMS is software that enables users to create, store, retrieve, and manage data systematically.
- Relational DBMS (RDBMS) like MySQL and Oracle store data in structured tables with defined relationships.
- NoSQL DBMS like MongoDB store data as flexible JSON-like documents, suitable for unstructured data.

MySQL

- MySQL is an open-source relational database system widely used in web applications.
- It supports SQL (Structured Query Language) for data definition, manipulation, and control.
- Default port: 3306. Installation includes the MySQL Server and optionally MySQL Workbench as a GUI.

Oracle Database

- Oracle is an enterprise-grade RDBMS known for scalability, reliability, and advanced SQL features.
- Oracle SQL*Plus is the default command-line tool; Oracle SQL Developer provides a GUI interface.
- Default port: 1521. Oracle Express Edition (XE) is a free version suitable for educational use.

MongoDB

- MongoDB is a document-oriented NoSQL database that stores data as BSON (Binary JSON) documents.
- MongoDB Compass provides a graphical interface; the mongo shell allows command-line interaction.
- Default port: 27017. It is schema-less, allowing flexible document structures within collections.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO1	3	2	1	3	–	1	2	2	2

Legend: 3 = High | 2 = Medium | 1 = Low | – = No Mapping

5. Lab Session Outcomes (LSOs)

After completing this practical, the student will be able to:

- Download and install MySQL/Oracle/MongoDB on a Windows or Linux system.
- Configure the database server and set up administrative credentials.
- Start and stop database services from the operating system.
- Connect to the database server using a command-line or GUI tool.
- Verify the installation by executing a basic query or command.

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i3 or above, 4 GB RAM minimum
2	Operating System	Windows 10 / Ubuntu 20.04 or above
3	Database Software	MySQL 8.x / Oracle 21c XE / MongoDB 6.x
4	GUI Tool	MySQL Workbench / Oracle SQL Developer / MongoDB Compass

7. Suggested Procedure

1. Download the installer for MySQL / Oracle XE / MongoDB from the official website.
2. Run the installer with administrator/root privileges.
3. Select 'Developer Default' or 'Complete' installation type.
4. Configure the root/admin password during installation setup.
5. Set the database service to start automatically with the system.
6. Complete the installation and note the default port number.
7. Open MySQL Workbench / Oracle SQL Developer / MongoDB Compass.
8. Create a new connection using localhost, default port, and root credentials.
9. Test the connection to ensure the server is running correctly.
10. Execute a basic command (e.g., SHOW DATABASES; or db.version()) to verify.
11. Record the output in the observation table.
12. Document the installation steps in the lab journal.

8. Program Code / SQL Script

```
-- MySQL: Verify Installation  
SHOW DATABASES;
```

```
-- Oracle: Verify Installation (SQL*Plus)  
SELECT * FROM v$version;
```

```
-- MongoDB: Verify Installation (mongo shell)  
db.version()  
show dbs
```

9. Sample Output / Observations

Sr.	Step / Operation Performed	Output / Behaviour
1	Launch database installer	Installer wizard opens successfully
2	Configure root password	Password accepted; service configured
3	Start the database service	Service started on default port
4	Connect via GUI/CLI tool	Connection established successfully
5	Execute SHOW DATABASES / show dbs	Default databases listed on screen
6	Verify version using SQL command	Version number displayed correctly
7	Close and restart service	Service restarts without error

10. Result

The database management system (MySQL / Oracle / MongoDB) was successfully installed and configured on the system. The server was started, a connection was established, and basic verification commands were executed without errors.

11. Interpretation of Results

- The successful connection confirmed that the database server is running on the expected port and accepting client requests.
- Default system databases (information_schema, mysql, etc.) were listed, indicating a complete installation.
- The GUI tool connected without errors, confirming correct host, port, and credential settings.
- Restarting the service verified that the database is configured to run persistently as a system service.

12. Precautions

- Always run the installer with administrator/root privileges to avoid permission errors.
- Note and securely store the root password configured during installation.
- Ensure that the default port (3306/1521/27017) is not blocked by the system firewall.
- Do not install multiple conflicting database servers on the same port without reconfiguring.

13. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Explain the difference between MySQL, Oracle, and MongoDB in terms of data storage.		✓	
2	If the database service fails to start, what steps would you follow to diagnose the issue?			✓

Practical No: 2.1

1. Aim / Practical Statement

Exercise on creating tables, inserting records, and updating records

2. Practical Significance

Creating tables and manipulating records are fundamental DDL and DML operations in any relational database. This practical enables students to define database structures and populate them with data, forming the basis for all subsequent query operations.

3. Relevant Theory

CREATE TABLE Statement

- CREATE TABLE defines a new table with column names, data types, and optional constraints.
- Syntax: CREATE TABLE table_name (col1 datatype constraint, col2 datatype, ...);
- Common data types: INT, VARCHAR(n), DATE, FLOAT, CHAR(n), NUMBER(p,s) in Oracle.

INSERT INTO Statement

- INSERT INTO adds new rows (records) into a table.
- Syntax: INSERT INTO table_name (col1, col2) VALUES (val1, val2);
- String values are enclosed in single quotes; numeric values are inserted directly.

UPDATE Statement

- UPDATE modifies existing records in a table based on a condition.
- Syntax: UPDATE table_name SET col1 = val1 WHERE condition;
- Omitting the WHERE clause updates ALL rows in the table — use with caution.

Primary Key Constraint

- PRIMARY KEY uniquely identifies each record in a table; it cannot be NULL or duplicate.
- Defined as: col_name datatype PRIMARY KEY or as a table-level constraint.
- Each table can have only one primary key.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO2	3	3	3	3	–	1	2	2	3

Legend: 3 = High | 2 = Medium | 1 = Low | – = No Mapping

5. Lab Session Outcomes (LSOs)

After completing this practical, the student will be able to:

- Write and execute a CREATE TABLE statement with appropriate data types.

- Insert multiple records into a table using INSERT INTO statements.
- Update specific records using the UPDATE statement with a WHERE condition.
- Verify table structure using DESC or DESCRIBE command.
- Use SELECT * to confirm inserted and updated data.

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i3 or above, 4 GB RAM minimum
2	Operating System	Windows 10 / Ubuntu 20.04 or above
3	Database Software	MySQL 8.x / Oracle 21c XE
4	IDE / Editor	MySQL Workbench / Oracle SQL Developer / Command Prompt

7. Suggested Procedure

1. Open MySQL Workbench or Oracle SQL Developer and connect to the database server.
2. Create a new database/schema: CREATE DATABASE CollegeDB; USE CollegeDB;
3. Write a CREATE TABLE statement for a STUDENT table with appropriate columns.
4. Execute the CREATE TABLE statement and verify using DESC STUDENT;
5. Write INSERT INTO statements to add at least 5 student records.
6. Execute the INSERT statements and check for successful insertion.
7. Execute SELECT * FROM STUDENT; to view all inserted records.
8. Write an UPDATE statement to modify one or more records based on a condition.
9. Execute the UPDATE statement and verify the change using SELECT.
10. Record actual outputs in the observation table.
11. Write the result and interpretation in the lab journal.
12. Answer viva-voce questions as asked by the faculty.

8. Program Code / SQL Script

-- Create database and table

```
CREATE DATABASE CollegeDB;
USE CollegeDB;
```

```
CREATE TABLE STUDENT (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Branch VARCHAR(30),
    Semester INT,
    Marks FLOAT
);
```

-- Insert records

```
INSERT INTO STUDENT VALUES (101, 'Amit Kumar', 'CSE', 4, 78.5);
INSERT INTO STUDENT VALUES (102, 'Priya Singh', 'CSE', 4, 85.0);
INSERT INTO STUDENT VALUES (103, 'Rahul Verma', 'ME', 4, 62.0);
INSERT INTO STUDENT VALUES (104, 'Neha Gupta', 'EE', 4, 90.5);
INSERT INTO STUDENT VALUES (105, 'Rohan Sharma', 'CSE', 4, 74.0);
```

```
-- Verify inserted records
SELECT * FROM STUDENT;

-- Update a record
UPDATE STUDENT SET Marks = 80.0 WHERE StudentID = 103;

-- Verify update
SELECT * FROM STUDENT WHERE StudentID = 103;
```

9. Sample Output / Observations

Sr.	Step / Operation Performed	Output / Behaviour
1	Execute CREATE TABLE statement	Table STUDENT created successfully
2	Execute DESC STUDENT;	5 columns displayed with data types
3	Insert 5 student records	5 rows inserted, 0 errors
4	SELECT * FROM STUDENT;	All 5 records displayed correctly
5	Execute UPDATE statement	1 row affected; Marks updated to 80.0
6	SELECT with WHERE after UPDATE	Updated record shows Marks = 80.0
7	Test PRIMARY KEY violation	Error: Duplicate entry — constraint enforced

10. Result

A STUDENT table was successfully created with a primary key and appropriate data types. Five records were inserted and one record was updated using a WHERE condition. All outputs matched the expected results.

11. Interpretation of Results

- The CREATE TABLE statement correctly defined the table schema, and DESC confirmed the structure without errors.
- All five INSERT statements executed successfully, confirming that data types and constraints were correctly specified.
- The UPDATE statement modified only the targeted record (StudentID = 103), demonstrating the importance of the WHERE clause.
- Attempting to insert a duplicate primary key generated an error, confirming that the PRIMARY KEY constraint is enforced.

12. Precautions

- Always use a WHERE clause with UPDATE to avoid accidentally modifying all rows in the table.
- Ensure that data types in INSERT values match the column definitions to avoid type mismatch errors.
- Verify the table structure using DESC before inserting records to avoid column order mistakes.

- Use COMMIT; after DML operations in Oracle to permanently save the changes.

13. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Explain the difference between CHAR and VARCHAR data types.		✓	
2	Write a query to update the branch of all students in Semester 4 to 'IT'.			✓
3	Design a table for storing employee records with at least 6 appropriate columns.			✓

Legend: R= Remember | U=Understand | A: Apply

Practical No: 3.1

1. Aim / Practical Statement

Exercise on modifying the structure of the table

2. Practical Significance

In real-world database systems, table structures often need to be modified after initial creation to accommodate new requirements. The ALTER TABLE command allows modification of an existing table without losing its data.

3. Relevant Theory

ALTER TABLE Statement

- ALTER TABLE is a DDL command used to change the structure of an existing table.
- It can add, drop, or modify columns, and add or drop constraints.
- Syntax: ALTER TABLE table_name ADD | MODIFY | DROP COLUMN col_name datatype;

Adding a Column

- New columns are added at the end of the table by default.
- Syntax (MySQL): ALTER TABLE table_name ADD column_name datatype;
- The new column is NULL for all existing rows unless a DEFAULT value is specified.

Modifying a Column

- MODIFY (Oracle) or MODIFY COLUMN (MySQL) changes the data type or size of an existing column.
- Reducing column size may fail if existing data exceeds the new size.
- Syntax: ALTER TABLE tbl MODIFY COLUMN col_name new_datatype;

Dropping a Column

- DROP COLUMN permanently removes a column and all its data from the table.
- Syntax: ALTER TABLE table_name DROP COLUMN column_name;
- A table must retain at least one column; dropping the only column is not allowed.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO2	3	3	3	3	–	1	2	2	3

Legend: 3 = High | 2 = Medium | 1 = Low | – = No Mapping

5. Lab Session Outcomes (LSOs)

After completing this practical, the student will be able to:

- Add a new column to an existing table using ALTER TABLE ADD.
- Modify the data type or size of an existing column using ALTER TABLE MODIFY.
- Drop an unwanted column from a table using ALTER TABLE DROP COLUMN.
- Rename a column using ALTER TABLE RENAME COLUMN (MySQL 8.0+).
- Verify structural changes using the DESC command.

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i3 or above, 4 GB RAM minimum
2	Operating System	Windows 10 / Ubuntu 20.04 or above
3	Database Software	MySQL 8.x / Oracle 21c XE
4	IDE / Editor	MySQL Workbench / Oracle SQL Developer

7. Suggested Procedure

1. Open the SQL editor and connect to the CollegeDB database.
2. Use DESC STUDENT; to view the existing structure of the table.
3. Write an ALTER TABLE statement to add a new column (e.g., Email VARCHAR(100)).
4. Execute the ADD statement and verify using DESC STUDENT;
5. Write an ALTER TABLE statement to modify the Marks column (e.g., change to DECIMAL(5,2)).
6. Execute the MODIFY statement and verify the change using DESC.
7. Write an ALTER TABLE statement to drop the Email column.
8. Execute the DROP COLUMN statement and verify using DESC.
9. Rename the Name column to StudentName using RENAME COLUMN.
10. Verify all changes using DESC STUDENT; and SELECT * FROM STUDENT;
11. Record actual outputs in the observation table.
12. Write results and interpretation in the lab journal.

8. Program Code / SQL Script

```
-- View existing structure
DESC STUDENT;

-- Add a new column
ALTER TABLE STUDENT ADD Email VARCHAR(100);

-- Verify
DESC STUDENT;

-- Modify column data type
ALTER TABLE STUDENT MODIFY COLUMN Marks DECIMAL(5,2);

-- Verify
```

```

DESC STUDENT;

-- Drop a column
ALTER TABLE STUDENT DROP COLUMN Email;

-- Verify
DESC STUDENT;

-- Rename a column (MySQL 8.0+)
ALTER TABLE STUDENT RENAME COLUMN Name TO StudentName;

-- Final structure
DESC STUDENT;
SELECT * FROM STUDENT;

```

9. Sample Output / Observations

Sr.	Step / Operation Performed	Output / Behaviour
1	DESC STUDENT before ALTER	5 columns shown as originally defined
2	ADD Email VARCHAR(100)	Email column added; 6 columns now visible
3	MODIFY Marks to DECIMAL(5,2)	Column type changed successfully
4	DROP COLUMN Email	Email removed; back to 5 columns
5	RENAME Name to StudentName	Column renamed; data preserved
6	SELECT * after all alterations	All 5 original rows intact with updated structure
7	Attempt to drop a non-existent column	Error: Unknown column — constraint enforced

10. Result

The ALTER TABLE command was used successfully to add, modify, drop, and rename columns in the STUDENT table. All structural changes were reflected immediately in the DESC output, and existing data remained intact throughout.

11. Interpretation of Results

- Adding a new column appended it at the end of the table with NULL values for all existing rows.
- Modifying the Marks column from FLOAT to DECIMAL(5,2) adjusted its precision without data loss.
- Dropping the Email column confirmed that ALTER TABLE can permanently remove column definitions and data.
- Renaming a column using RENAME COLUMN showed that column metadata can be changed without affecting stored data values.

12. Precautions

- Take a backup of table data before performing structural changes using ALTER TABLE.
- Reducing column size (e.g., VARCHAR(100) to VARCHAR(10)) may fail if existing data is longer than the new limit.
- Dropping a column permanently removes all data stored in it — verify before executing DROP COLUMN.
- In Oracle, the syntax for modifying a column uses MODIFY without the COLUMN keyword.

13. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Explain the difference between DROP TABLE and ALTER TABLE DROP COLUMN.		✓	
2	Write a query to add columns Age (INT) and City (VARCHAR(50)) to the EMPLOYEE table.			✓

Legend: R= Remember | U=Understand | A: Apply

Practical No: 4.1

1. Aim / Practical Statement

Exercise on SELECT command

2. Practical Significance

The SELECT command is the most frequently used SQL statement, allowing retrieval of specific data from one or more tables. Mastering SELECT is essential for data analysis, report generation, and application development.

3. Relevant Theory

SELECT Statement

- SELECT retrieves data from one or more tables based on specified columns and conditions.
- Syntax: SELECT col1, col2 FROM table_name;
- SELECT * retrieves all columns; selecting specific columns improves performance.

Column Aliases

- AS keyword creates a temporary alias for a column in the result set.
- Syntax: SELECT col_name AS alias_name FROM table;
- Aliases improve result readability, especially for calculated columns.

DISTINCT Keyword

- DISTINCT eliminates duplicate rows from the query result.
- Syntax: SELECT DISTINCT col_name FROM table_name;
- Useful for listing unique values such as distinct departments or cities.

Arithmetic Expressions in SELECT

- SQL supports arithmetic operators (+, -, *, /) in SELECT for computed columns.
- Example: SELECT Name, Marks * 1.1 AS IncreasedMarks FROM STUDENT;
- The computed column appears in the result but does not modify the stored data.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO3	3	3	3	3	–	1	2	3	3

Legend: 3 = High | 2 = Medium | 1 = Low | – = No Mapping

5. Lab Session Outcomes (LSOs)

After completing this practical, the student will be able to:

- Retrieve all records from a table using SELECT *.

- Select specific columns from a table using SELECT col1, col2.
- Use DISTINCT to retrieve unique values from a column.
- Apply column aliases using the AS keyword.
- Perform arithmetic calculations within a SELECT statement.

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i3 or above, 4 GB RAM minimum
2	Operating System	Windows 10 / Ubuntu 20.04 or above
3	Database Software	MySQL 8.x / Oracle 21c XE
4	IDE / Editor	MySQL Workbench / Oracle SQL Developer

7. Suggested Procedure

1. Open the SQL editor and connect to the CollegeDB database.
2. Execute SELECT * FROM STUDENT; to retrieve all records and columns.
3. Execute a query to select only StudentID, StudentName, and Marks columns.
4. Use a column alias to display Marks as 'Score' in the result.
5. Execute SELECT DISTINCT Branch FROM STUDENT; to find unique branches.
6. Write a query to calculate and display Marks multiplied by 1.1 as 'BonusMarks'.
7. Observe and record the result set for each query.
8. Combine multiple features: DISTINCT, alias, and arithmetic in one query.
9. Record actual outputs in the observation table.
10. Compare actual output with expected output.
11. Write results in the lab journal.
12. Answer viva-voce questions as asked by the faculty.

8. Program Code / SQL Script

```
-- Retrieve all records
SELECT * FROM STUDENT;

-- Select specific columns
SELECT StudentID, StudentName, Marks FROM STUDENT;

-- Column alias
SELECT StudentName, Marks AS Score FROM STUDENT;

-- DISTINCT values
SELECT DISTINCT Branch FROM STUDENT;

-- Arithmetic expression
SELECT StudentName, Marks, Marks * 1.1 AS BonusMarks FROM STUDENT;

-- Combined query
SELECT DISTINCT Branch, Semester FROM STUDENT;
```

9. Sample Output / Observations

Sr.	Step / Operation Performed	Output / Behaviour
1	SELECT * FROM STUDENT;	All 5 rows and all columns displayed
2	SELECT StudentName, Marks FROM STUDENT;	2 columns shown for all 5 rows
3	SELECT Marks AS Score	Column header shows 'Score' instead of 'Marks'
4	SELECT DISTINCT Branch	3 unique branches returned (CSE, ME, EE)
5	Marks * 1.1 AS BonusMarks	Calculated column displayed; original data unchanged
6	Verify original table after queries	SELECT * shows original Marks values unchanged
7	DISTINCT on Semester column	Single value '4' returned

10. Result

Various forms of the SELECT command were executed successfully. Specific columns were retrieved, aliases were applied, DISTINCT values were obtained, and arithmetic expressions produced computed result columns. The original table data remained unmodified.

11. Interpretation of Results

- SELECT * returned all rows and columns, confirming all inserted data is correctly stored.
- Column aliases using AS improved readability of the result without altering table structure.
- DISTINCT Branch returned only 3 unique branch names, confirming it eliminates duplicate values.
- The arithmetic expression (Marks * 1.1) demonstrated that SQL can perform calculations within the SELECT clause.

12. Precautions

- Use SELECT * only for exploration; in production, always specify required columns to save bandwidth.
- Remember that arithmetic operations in SELECT do not modify stored data in the table.
- Column aliases with spaces must be enclosed in backticks (MySQL) or double quotes (Oracle).
- DISTINCT applies to the entire selected column set, not individual columns independently.

13. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Explain why SELECT * is not recommended in production applications.		✓	
2	Write a query to display StudentName and a calculated column showing Marks increased by 10%.			✓

Legend: R= Remember | U=Understand | A: Apply

Practical No: 5.1

1. Aim / Practical Statement

Exercise on querying the table using clauses: WHERE, ORDER BY, IN, AND, OR, NOT

2. Practical Significance

Filtering and ordering data are essential operations in database querying. The WHERE clause and related logical operators allow precise data retrieval, while ORDER BY organizes results for better analysis and reporting.

3. Relevant Theory

WHERE Clause

- WHERE filters rows returned by a query based on a specified condition.
- Syntax: SELECT * FROM table WHERE condition;
- Comparison operators used: =, !=, <, >, <=, >=.

Logical Operators: AND, OR, NOT

- AND: Both conditions must be true for a row to be included.
- OR: At least one condition must be true.
- NOT: Negates a condition (e.g., NOT IN, NOT LIKE).

IN Operator

- IN checks whether a column value matches any value in a given list.
- Syntax: WHERE col_name IN (val1, val2, val3);
- Equivalent to multiple OR conditions; more readable for long lists.

ORDER BY Clause

- ORDER BY sorts the result set in ascending (ASC, default) or descending (DESC) order.
- Syntax: SELECT * FROM table ORDER BY col_name ASC|DESC;
- Multiple columns can be specified for multi-level sorting.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO3	3	3	3	3	–	1	2	3	3

Legend: 3 = High | 2 = Medium | 1 = Low | – = No Mapping

5. Lab Session Outcomes (LSOs)

After completing this practical, the student will be able to:

- Filter records using the WHERE clause with comparison operators.

- Combine multiple conditions using AND and OR operators.
- Negate conditions using the NOT operator.
- Use the IN operator to match multiple values in a list.
- Sort query results using ORDER BY in ASC and DESC order.

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i3 or above, 4 GB RAM minimum
2	Operating System	Windows 10 / Ubuntu 20.04 or above
3	Database Software	MySQL 8.x / Oracle 21c XE
4	IDE / Editor	MySQL Workbench / Oracle SQL Developer

7. Suggested Procedure

1. Open the SQL editor and connect to CollegeDB.
2. Write and execute a WHERE query to retrieve students with Marks > 75.
3. Write a query combining AND: Branch = 'CSE' AND Marks > 70.
4. Write a query using OR: Branch = 'CSE' OR Branch = 'ME'.
5. Write a query using NOT: NOT Branch = 'EE'.
6. Write a query using IN: Branch IN ('CSE', 'ME').
7. Execute SELECT * FROM STUDENT ORDER BY Marks DESC;
8. Execute ORDER BY with multiple columns: ORDER BY Branch ASC, Marks DESC.
9. Record all outputs in the observation table.
10. Compare with expected results.
11. Write the result and interpretation in the lab journal.
12. Answer viva-voce questions as asked by the faculty.

8. Program Code / SQL Script

```
-- WHERE with comparison
SELECT * FROM STUDENT WHERE Marks > 75;

-- AND operator
SELECT * FROM STUDENT WHERE Branch = 'CSE' AND Marks > 70;

-- OR operator
SELECT * FROM STUDENT WHERE Branch = 'CSE' OR Branch = 'ME';

-- NOT operator
SELECT * FROM STUDENT WHERE NOT Branch = 'EE';

-- IN operator
SELECT * FROM STUDENT WHERE Branch IN ('CSE', 'ME');

-- ORDER BY (descending)
SELECT * FROM STUDENT ORDER BY Marks DESC;
```

```
-- Multi-column ORDER BY
SELECT * FROM STUDENT ORDER BY Branch ASC, Marks DESC;
```

9. Sample Output / Observations

Sr.	Step / Operation Performed	Output / Behaviour
1	WHERE Marks > 75	3 rows returned (Marks 78.5, 85.0, 90.5)
2	Branch = 'CSE' AND Marks > 70	3 CSE students with Marks above 70 returned
3	Branch = 'CSE' OR Branch = 'ME'	4 rows returned (3 CSE + 1 ME)
4	NOT Branch = 'EE'	4 rows returned, EE student excluded
5	Branch IN ('CSE', 'ME')	Same result as OR query — 4 rows
6	ORDER BY Marks DESC	Rows sorted highest to lowest marks
7	ORDER BY Branch ASC, Marks DESC	EE first, then ME, then CSE; within each, highest Marks first

10. Result

All filtering clauses (WHERE, AND, OR, NOT, IN) and the ORDER BY clause were executed successfully. Results matched expected outputs confirming correct logic and query syntax.

11. Interpretation of Results

- The WHERE clause precisely filtered rows, showing only students satisfying the given condition.
- AND narrowed results further while OR broadened them, demonstrating the complementary nature of logical operators.
- IN operator produced the same result as an equivalent OR clause, confirming its role as a cleaner alternative.
- ORDER BY sorted records predictably, and multi-column sorting showed correct priority-based ordering.

12. Precautions

- Always use single quotes for string values in WHERE conditions (e.g., WHERE Branch = 'CSE').
- Operator precedence: AND is evaluated before OR; use parentheses to override default precedence.
- NOT negates the entire condition — verify the logic before using NOT with complex conditions.
- ORDER BY does not modify the stored data; it only affects the display order of the result set.

13. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Explain how ORDER BY ASC and DESC differ with an example.		✓	
2	Write a query to retrieve all students from CSE with Marks between 70 and 90.			✓

Legend: R= Remember | U=Understand | A: Apply

Practical No: 6.1

1. Aim / Practical Statement

Exercise on various group functions

2. Practical Significance

Group functions (aggregate functions) are essential for summarising and analyzing large datasets. They allow computation of statistics like count, sum, average, minimum, and maximum across groups of records, which is fundamental to business reporting.

3. Relevant Theory

Aggregate Functions

- Aggregate functions operate on a set of rows and return a single summarized value.
- Key functions: COUNT(), SUM(), AVG(), MIN(), MAX().
- They are used in the SELECT clause, often with GROUP BY and HAVING.

COUNT() Function

- COUNT(*) counts all rows including NULLs; COUNT(col) ignores NULL values.
- Syntax: SELECT COUNT(*) FROM table_name;
- Often used with GROUP BY to count records in each group.

GROUP BY Clause

- GROUP BY groups rows with the same value in a specified column into summary rows.
- All non-aggregated columns in SELECT must appear in GROUP BY.
- Syntax: SELECT col, AGG_FUNC(col) FROM table GROUP BY col;

HAVING Clause

- HAVING filters groups produced by GROUP BY based on a condition on aggregate values.
- Unlike WHERE (filters rows), HAVING filters groups.
- Syntax: SELECT col, COUNT(*) FROM table GROUP BY col HAVING COUNT(*) > 1;

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO4	3	3	2	3	–	1	2	2	3

Legend: 3 = High | 2 = Medium | 1 = Low | – = No Mapping

5. Lab Session Outcomes (LSOs)

After completing this practical, the student will be able to:

- Apply COUNT(), SUM(), AVG(), MIN(), MAX() functions on table columns.

- Use GROUP BY to group records and compute group-level aggregates.
- Apply the HAVING clause to filter grouped results.
- Combine GROUP BY with ORDER BY for sorted aggregate reports.
- Distinguish between WHERE and HAVING in filtering behaviour.

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i3 or above, 4 GB RAM minimum
2	Operating System	Windows 10 / Ubuntu 20.04 or above
3	Database Software	MySQL 8.x / Oracle 21c XE
4	IDE / Editor	MySQL Workbench / Oracle SQL Developer

7. Suggested Procedure

1. Open the SQL editor and connect to CollegeDB.
2. Execute SELECT COUNT(*) FROM STUDENT; to count total students.
3. Execute queries for AVG(Marks), MAX(Marks), MIN(Marks), SUM(Marks).
4. Write GROUP BY query: count students per branch.
5. Write GROUP BY query: average marks per branch.
6. Apply HAVING to show only branches where AVG(Marks) > 75.
7. Combine GROUP BY with ORDER BY AVG(Marks) DESC.
8. Record all outputs in the observation table.
9. Compare actual with expected outputs.
10. Write results in the lab journal.
11. Answer viva-voce questions as asked by the faculty.

8. Program Code / SQL Script

```
-- Total count
SELECT COUNT(*) AS TotalStudents FROM STUDENT;

-- Aggregate functions
SELECT AVG(Marks) AS AvgMarks, MAX(Marks) AS HighestMarks,
       MIN(Marks) AS LowestMarks, SUM(Marks) AS TotalMarks
FROM STUDENT;

-- Count per branch
SELECT Branch, COUNT(*) AS StudentCount FROM STUDENT GROUP BY Branch;

-- Avg marks per branch
SELECT Branch, AVG(Marks) AS AvgMarks FROM STUDENT GROUP BY Branch;

-- HAVING clause
SELECT Branch, AVG(Marks) AS AvgMarks FROM STUDENT
GROUP BY Branch
HAVING AVG(Marks) > 75;
```

```
-- GROUP BY + ORDER BY
SELECT Branch, AVG(Marks) AS AvgMarks FROM STUDENT
GROUP BY Branch
ORDER BY AvgMarks DESC;
```

9. Sample Output / Observations

Sr.	Step / Operation Performed	Output / Behaviour
1	COUNT(*) FROM STUDENT	5 total students
2	AVG(Marks)	78.0 (sum of all marks / 5)
3	MAX(Marks) and MIN(Marks)	90.5 and 62.0 respectively
4	GROUP BY Branch — COUNT	CSE:3, ME:1, EE:1
5	GROUP BY Branch — AVG(Marks)	CSE:79.17, ME:80.0, EE:90.5
6	HAVING AVG(Marks) > 75	All 3 branches shown (all avg > 75)
7	ORDER BY AvgMarks DESC	EE (90.5), ME (80.0), CSE (79.17)

10. Result

All aggregate functions (COUNT, AVG, MAX, MIN, SUM) were applied successfully. GROUP BY grouped records by branch and HAVING filtered groups based on aggregate conditions. Results were verified and matched expected values.

11. Interpretation of Results

- COUNT(*) returned 5, confirming the total number of records in the STUDENT table.
- AVG, MAX, and MIN provided useful statistical summaries of student performance.
- GROUP BY Branch partitioned the data correctly, enabling per-branch analysis.
- HAVING filtered groups post-aggregation, which is not possible with a WHERE clause.

12. Precautions

- Do not use WHERE to filter aggregate results — use HAVING for conditions on grouped data.
- All non-aggregated columns in the SELECT clause must be included in the GROUP BY clause.
- COUNT(column_name) ignores NULL values; use COUNT(*) to count all rows including NULLs.
- Round AVG results if required for clarity: SELECT ROUND(AVG(Marks), 2) AS AvgMarks;

13. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Name the five SQL aggregate functions.	✓		

S. N o.	Viva-Voce Question	R	U	A
2	Explain the order of execution: WHERE → GROUP BY → HAVING → SELECT.		✓	
3	Write a query to display each branch and its highest marks among students with Marks > 60.			✓

Legend: R= Remember | U=Understand | A: Apply

Practical No: 7.1

1. Aim / Practical Statement

Exercise on number functions, character functions, conversion functions, and date functions

2. Practical Significance

SQL built-in functions provide powerful tools for transforming, formatting, and processing data directly within queries. Mastery of these functions reduces the need for application-level data processing and enables efficient server-side computation.

3. Relevant Theory

Number Functions

- **ROUND(n, d)**: rounds number n to d decimal places.
- **FLOOR(n)**: returns the largest integer less than or equal to n.
- **CEIL(n) / CEILING(n)**: returns the smallest integer greater than or equal to n.
- **MOD(n, m)**: returns the remainder of n divided by m.

Character / String Functions

- **UPPER(str) / LOWER(str)**: converts string to uppercase or lowercase.
- **LENGTH(str)**: returns the number of characters in the string.
- **SUBSTRING(str, start, len)**: extracts a part of the string.
- **CONCAT(str1, str2)**: joins two or more strings together.

Conversion Functions

- **CAST(value AS datatype)**: converts a value to the specified data type.
- **CONVERT(value, datatype)**: MySQL-specific function for type conversion.
- **DATE_FORMAT(date, format)**: formats a DATE value as a string in MySQL.

Date Functions

- **NOW() / SYSDATE**: returns the current date and time.
- **CURDATE()**: returns the current date (date only, no time).
- **DATEDIFF(date1, date2)**: returns the number of days between two dates.
- **DATE_ADD(date, INTERVAL n UNIT)**: adds a time interval to a date.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO4	3	3	2	3	–	1	2	2	3

Legend: 3 = High | 2 = Medium | 1 = Low | – = No Mapping

5. Lab Session Outcomes (LSOs)

After completing this practical, the student will be able to:

- Apply number functions (ROUND, FLOOR, CEIL, MOD) in SQL queries.
- Use character functions (UPPER, LOWER, LENGTH, CONCAT, SUBSTRING) on string columns.
- Convert data types using CAST and CONVERT functions.
- Use date functions (NOW, CURDATE, DATEDIFF, DATE_ADD) for date computations.
- Combine multiple functions in a single SELECT statement.

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i3 or above, 4 GB RAM minimum
2	Operating System	Windows 10 / Ubuntu 20.04 or above
3	Database Software	MySQL 8.x / Oracle 21c XE
4	IDE / Editor	MySQL Workbench / Oracle SQL Developer

7. Suggested Procedure

1. Open the SQL editor and connect to CollegeDB.
2. Execute ROUND, FLOOR, CEIL, and MOD on numeric values.
3. Apply UPPER, LOWER, LENGTH, and CONCAT on the StudentName column.
4. Use SUBSTRING to extract part of the StudentName.
5. Use CAST to convert Marks (DECIMAL) to INT.
6. Execute NOW() and CURDATE() to retrieve current date/time.
7. Compute DATEDIFF between two given dates.
8. Use DATE_ADD to add 30 days to the current date.
9. Record all outputs in the observation table.
10. Compare actual with expected outputs.
11. Write results in the lab journal.
12. Answer viva-voce questions.

8. Program Code / SQL Script

-- Number Functions

```
SELECT ROUND(78.567, 2), FLOOR(78.9), CEIL(78.1), MOD(10, 3);
```

-- Character Functions

```
SELECT UPPER(StudentName), LOWER(StudentName),  
       LENGTH(StudentName), CONCAT(StudentName, ' - ', Branch)  
FROM STUDENT;
```

-- SUBSTRING

```
SELECT StudentName, SUBSTRING(StudentName, 1, 5) AS ShortName  
FROM STUDENT;
```

-- Conversion Functions

```
SELECT StudentName, Marks, CAST(Marks AS UNSIGNED) AS MarksInt
```

FROM STUDENT;

-- Date Functions

SELECT NOW(), CURDATE();

SELECT DATEDIFF('2024-12-31', '2024-01-01') AS DaysDiff;

SELECT DATE_ADD(CURDATE(), INTERVAL 30 DAY) AS DateAfter30Days;

9. Sample Output / Observations

Sr.	Step / Operation Performed	Output / Behaviour
1	ROUND(78.567, 2)	78.57
2	FLOOR(78.9) / CEIL(78.1)	78 / 79
3	MOD(10, 3)	1
4	UPPER / LOWER on StudentName	All-caps and all-lowercase names displayed
5	LENGTH(StudentName)	Character counts for each name
6	CONCAT(StudentName, ' - ', Branch)	e.g., 'Amit Kumar - CSE'
7	DATEDIFF('2024-12-31', '2024-01-01')	365 days

10. Result

Number, character, conversion, and date functions were applied successfully in SQL queries. Each function produced the expected output, demonstrating SQL's capability for in-query data transformation.

11. Interpretation of Results

- ROUND, FLOOR, and CEIL demonstrated three different approaches to numeric rounding, each with distinct behaviour.
- UPPER and LOWER are useful for case-insensitive comparisons and standardised data display.
- DATEDIFF computed the exact day count between two dates, useful for age and tenure calculations.
- DATE_ADD showed how SQL can perform date arithmetic directly, eliminating the need for application-level calculation.

12. Precautions

- Be aware that Oracle uses TRUNC instead of FLOOR, and ADD_MONTHS instead of DATE_ADD for month intervals.
- SUBSTRING indexing starts at 1 in SQL, not 0 as in most programming languages.
- CAST to a smaller data type may truncate data — verify that the conversion is safe before applying.
- NOW() returns timestamp (date + time); CURDATE() returns date only — choose based on requirement.

13. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Explain the difference between NOW() and CURDATE().		✓	
2	Write a query to display each student's name in uppercase along with their marks rounded to 0 decimals.			✓

Legend: R= Remember| U=Understand | A: Apply

Practical No: 8.1

1. Aim / Practical Statement

Exercise on set operators

2. Practical Significance

Set operators allow combining result sets from multiple queries, enabling complex data retrieval that mirrors set theory concepts. They are essential in scenarios requiring comparison, merging, or differentiation of data from multiple tables or queries.

3. Relevant Theory

UNION Operator

- UNION combines results from two SELECT queries, removing duplicate rows.
- Both SELECT statements must have the same number of columns with compatible data types.
- UNION ALL retains all rows including duplicates — it is faster than UNION.

INTERSECT Operator

- INTERSECT returns only the rows that are common to both SELECT queries.
- MySQL does not natively support INTERSECT (use INNER JOIN as an alternative).
- Oracle and SQL Server support INTERSECT directly.

EXCEPT / MINUS Operator

- EXCEPT (SQL standard) / MINUS (Oracle) returns rows from the first query not found in the second.
- MySQL does not support EXCEPT natively; use LEFT JOIN with IS NULL as workaround.
- Useful for finding records that exist in one table but not another.

Rules for Set Operators

- Both queries must return the same number of columns.
- Corresponding columns must have compatible data types.
- Column names in the result are taken from the first SELECT query.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO3	3	3	3	3	–	1	2	3	3

Legend: 3 = High | 2 = Medium | 1 = Low | – = No Mapping

5. Lab Session Outcomes (LSOs)

After completing this practical, the student will be able to:

- Combine two queries using UNION and UNION ALL and observe the difference.
- Use INTERSECT to find common records between two queries.
- Apply EXCEPT / MINUS to find records unique to the first query.
- Understand and apply the column compatibility rule for set operators.
- Interpret and compare results of different set operators.

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i3 or above, 4 GB RAM minimum
2	Operating System	Windows 10 / Ubuntu 20.04 or above
3	Database Software	MySQL 8.x / Oracle 21c XE
4	IDE / Editor	MySQL Workbench / Oracle SQL Developer

7. Suggested Procedure

1. Open the SQL editor and connect to CollegeDB.
2. Create a second table ALUMNI with similar columns to STUDENT.
3. Insert records into ALUMNI (some overlapping with STUDENT).
4. Execute UNION query combining Branch from STUDENT and ALUMNI.
5. Execute UNION ALL and observe duplicate handling.
6. Simulate INTERSECT using INNER JOIN (MySQL) or direct INTERSECT (Oracle).
7. Simulate MINUS/EXCEPT using LEFT JOIN with NULL check or MINUS (Oracle).
8. Record all outputs in the observation table.
9. Compare actual with expected outputs.
10. Write results in the lab journal.
11. Answer viva-voce questions as asked by the faculty.

8. Program Code / SQL Script

```
-- Create second table
CREATE TABLE ALUMNI (
  AlumniID INT PRIMARY KEY,
  Name  VARCHAR(50),
  Branch VARCHAR(30),
  PassYear INT
);

INSERT INTO ALUMNI VALUES (201, 'Suresh Patel', 'CSE', 2022);
INSERT INTO ALUMNI VALUES (202, 'Kavya Mehta', 'ME', 2022);
INSERT INTO ALUMNI VALUES (203, 'Amit Kumar', 'CSE', 2022);

-- UNION (removes duplicates)
SELECT Branch FROM STUDENT
UNION
SELECT Branch FROM ALUMNI;

-- UNION ALL (keeps duplicates)
```

```
SELECT Branch FROM STUDENT
UNION ALL
SELECT Branch FROM ALUMNI;
```

```
-- INTERSECT (MySQL workaround using JOIN)
SELECT DISTINCT s.Branch
FROM STUDENT s
INNER JOIN ALUMNI a ON s.Branch = a.Branch;
```

```
-- MINUS / EXCEPT (MySQL workaround)
SELECT DISTINCT s.Branch
FROM STUDENT s
LEFT JOIN ALUMNI a ON s.Branch = a.Branch
WHERE a.Branch IS NULL;
```

9. Sample Output / Observations

Sr.	Step / Operation Performed	Output / Behaviour
1	UNION of Branch columns	3 unique branches: CSE, ME, EE
2	UNION ALL of Branch columns	8 rows total with duplicates included
3	INTERSECT (JOIN workaround)	CSE and ME — branches common to both tables
4	MINUS (LEFT JOIN workaround)	EE — branch only in STUDENT, not in ALUMNI
5	Column count mismatch test	Error: queries must have same number of columns
6	SELECT Name UNION SELECT PassYear	Error: incompatible data types
7	UNION ALL row count	5 (STUDENT) + 3 (ALUMNI) = 8 rows

10. Result

Set operators UNION, UNION ALL, and simulated INTERSECT and EXCEPT were executed successfully. UNION removed duplicates while UNION ALL retained them. The column compatibility rule was verified by triggering an intentional error.

11. Interpretation of Results

- UNION returned 3 unique branch names, confirming deduplication across both tables.
- UNION ALL returned 8 rows (5+3), demonstrating that all rows including duplicates are retained.
- The JOIN-based INTERSECT correctly identified CSE and ME as branches present in both tables.
- The LEFT JOIN IS NULL technique correctly identified EE as a branch exclusive to the STUDENT table.

12. Precautions

- Ensure both SELECT statements in a set operation return the same number of columns.
- Corresponding columns must have compatible (not necessarily identical) data types.
- UNION ALL is faster than UNION — use UNION only when duplicate removal is specifically required.
- In MySQL, INTERSECT and EXCEPT are not directly supported; use JOIN-based workarounds.

13. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Write a query to find all branch names that exist in STUDENT but not in ALUMNI.			✓

Legend: R= Remember | U=Understand | A: Apply

Practical No: 9.1

1. Aim / Practical Statement

Exercise on subqueries

2. Practical Significance

Subqueries enable multi-step retrieval by embedding one query within another. They are widely used in complex data retrieval scenarios such as finding records based on aggregate results or values from another table.

3. Relevant Theory

Subquery (Nested Query)

- A subquery is a SELECT statement embedded within another SQL statement.
- The inner query executes first; its result is passed to the outer query.
- Subqueries can appear in SELECT, FROM, WHERE, and HAVING clauses.

Single-Row Subquery

- Returns exactly one row; used with comparison operators: =, >, <, >=, <=.
- Example: WHERE Marks > (SELECT AVG(Marks) FROM STUDENT);
- If the inner query returns more than one row, an error occurs.

Multi-Row Subquery

- Returns multiple rows; used with IN, ANY, ALL operators.
- Example: WHERE Branch IN (SELECT DISTINCT Branch FROM ALUMNI);
- IN checks if a value matches any value in the subquery result set.

Correlated Subquery

- A correlated subquery references columns from the outer query.
- It is re-evaluated for each row processed by the outer query.
- Example: WHERE Marks > (SELECT AVG(Marks) FROM STUDENT s2 WHERE s2.Branch = s1.Branch).

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO3	3	3	3	3	–	1	2	3	3

Legend: 3 = High | 2 = Medium | 1 = Low | – = No Mapping

5. Lab Session Outcomes (LSOs)

After completing this practical, the student will be able to:

- Write single-row subqueries using comparison operators.
- Write multi-row subqueries using IN and NOT IN.
- Apply subqueries in the WHERE clause to filter based on aggregated results.
- Use correlated subqueries to compare rows against group-level statistics.
- Distinguish between subqueries and JOIN-based solutions.

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i3 or above, 4 GB RAM minimum
2	Operating System	Windows 10 / Ubuntu 20.04 or above
3	Database Software	MySQL 8.x / Oracle 21c XE
4	IDE / Editor	MySQL Workbench / Oracle SQL Developer

7. Suggested Procedure

1. Open the SQL editor and connect to CollegeDB.
2. Write a subquery to find students with marks above the overall average.
3. Write a subquery to find the student with maximum marks.
4. Write a multi-row subquery: branches in STUDENT that also appear in ALUMNI.
5. Write a NOT IN subquery to find branches in STUDENT not in ALUMNI.
6. Write a correlated subquery: students above their branch's average marks.
7. Execute each query and record outputs.
8. Compare actual with expected outputs.
9. Write results in the lab journal.
10. Answer viva-voce questions.

8. Program Code / SQL Script

```
-- Students with marks above overall average
SELECT StudentName, Marks FROM STUDENT
WHERE Marks > (SELECT AVG(Marks) FROM STUDENT);

-- Student with maximum marks
SELECT StudentName, Marks FROM STUDENT
WHERE Marks = (SELECT MAX(Marks) FROM STUDENT);

-- Multi-row subquery: branches in ALUMNI
SELECT * FROM STUDENT
WHERE Branch IN (SELECT DISTINCT Branch FROM ALUMNI);

-- NOT IN: branches not in ALUMNI
SELECT * FROM STUDENT
WHERE Branch NOT IN (SELECT DISTINCT Branch FROM ALUMNI);

-- Correlated subquery: above branch average
SELECT s1.StudentName, s1.Branch, s1.Marks
FROM STUDENT s1
```

```

WHERE s1.Marks > (
    SELECT AVG(s2.Marks)
    FROM STUDENT s2
    WHERE s2.Branch = s1.Branch
);

```

9. Sample Output / Observations

Sr.	Step / Operation Performed	Output / Behaviour
1	Marks > AVG subquery	3 students with marks above 78.0 average
2	Marks = MAX subquery	Neha Gupta (90.5) returned
3	Branch IN ALUMNI subquery	4 students from CSE and ME branches
4	Branch NOT IN ALUMNI	1 student from EE branch
5	Correlated subquery	Students above their own branch average identified
6	Subquery returning > 1 row with = operator	Error: subquery returns more than 1 row
7	Verify logic of correlated query manually	Results match manual calculation

10. Result

Single-row, multi-row, and correlated subqueries were executed and produced correct results. Subqueries demonstrated their ability to filter records based on dynamic, query-derived values.

11. Interpretation of Results

- The AVG subquery dynamically computed the threshold — no hardcoded value was needed.
- IN with subquery efficiently filtered students based on values from another table.
- NOT IN correctly identified branches exclusive to STUDENT, complementing the IN query.
- The correlated subquery compared each student's marks against their own branch average, demonstrating row-by-row evaluation.

12. Precautions

- Use single-row operators (=, >, <) only when the subquery is guaranteed to return one row.
- Use IN, ANY, ALL for multi-row subqueries to avoid 'subquery returns more than 1 row' errors.
- Correlated subqueries can be slow for large tables — consider JOIN-based alternatives for performance.
- Test inner queries independently first before embedding them in the outer query.

13. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Explain the difference between a subquery in WHERE vs. a subquery in FROM.		✓	
2	Write a subquery to find all students whose marks are below the minimum marks in the CSE branch.			✓
3	Rewrite the IN-based subquery using a JOIN and verify that the results match.			✓

Legend: R= Remember | U=Understand | A: Apply

Practical No: 10.1

1. Aim / Practical Statement

Exercise on joins

2. Practical Significance

Joins are the cornerstone of relational databases, enabling retrieval of related data from multiple tables. Understanding different join types is essential for designing and querying normalized databases in real-world applications.

3. Relevant Theory

INNER JOIN

- Returns only rows that have matching values in both tables.
- Syntax: `SELECT * FROM t1 INNER JOIN t2 ON t1.col = t2.col;`
- Non-matching rows from either table are excluded from the result.

LEFT JOIN (LEFT OUTER JOIN)

- Returns all rows from the left table and matching rows from the right table.
- Non-matching rows from the right table appear as NULL in the result.
- Syntax: `SELECT * FROM t1 LEFT JOIN t2 ON t1.col = t2.col;`

RIGHT JOIN (RIGHT OUTER JOIN)

- Returns all rows from the right table and matching rows from the left table.
- Non-matching rows from the left table appear as NULL in the result.
- Syntax: `SELECT * FROM t1 RIGHT JOIN t2 ON t1.col = t2.col;`

CROSS JOIN

- Returns the Cartesian product of both tables — every combination of rows.
- Result rows = rows in table 1 $\times$ rows in table 2.
- Rarely used in practice; useful for generating combinations.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO3	3	3	3	3	–	1	2	3	3

Legend: 3 = High | 2 = Medium | 1 = Low | – = No Mapping

5. Lab Session Outcomes (LSOs)

After completing this practical, the student will be able to:

- Create two related tables and populate them with data.

- Apply INNER JOIN to retrieve matching records from two tables.
- Apply LEFT JOIN to retrieve all records from the left table.
- Apply RIGHT JOIN to retrieve all records from the right table.
- Observe the effect of NULL values in outer join results.

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i3 or above, 4 GB RAM minimum
2	Operating System	Windows 10 / Ubuntu 20.04 or above
3	Database Software	MySQL 8.x / Oracle 21c XE
4	IDE / Editor	MySQL Workbench / Oracle SQL Developer

7. Suggested Procedure

1. Open SQL editor and connect to CollegeDB.
2. Create a DEPARTMENT table with DeptID and DeptName.
3. Add a DeptID foreign key column to the STUDENT table.
4. Update STUDENT records with appropriate DeptID values.
5. Execute INNER JOIN between STUDENT and DEPARTMENT.
6. Execute LEFT JOIN and observe NULL for unmatched departments.
7. Execute RIGHT JOIN and observe NULL for unmatched students.
8. Execute CROSS JOIN and observe the Cartesian product row count.
9. Record all outputs in the observation table.
10. Write results in the lab journal.
11. Answer viva-voce questions.

8. Program Code / SQL Script

```
-- Create DEPARTMENT table
CREATE TABLE DEPARTMENT (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);

INSERT INTO DEPARTMENT VALUES (1, 'Computer Science');
INSERT INTO DEPARTMENT VALUES (2, 'Mechanical');
INSERT INTO DEPARTMENT VALUES (3, 'Electrical');
INSERT INTO DEPARTMENT VALUES (4, 'Civil');

-- Add DeptID to STUDENT
ALTER TABLE STUDENT ADD DeptID INT;
UPDATE STUDENT SET DeptID = 1 WHERE Branch = 'CSE';
UPDATE STUDENT SET DeptID = 2 WHERE Branch = 'ME';
UPDATE STUDENT SET DeptID = 3 WHERE Branch = 'EE';

-- INNER JOIN
SELECT s.StudentName, s.Marks, d.DeptName
```

```
FROM STUDENT s
INNER JOIN DEPARTMENT d ON s.DeptID = d.DeptID;
```

```
-- LEFT JOIN
SELECT s.StudentName, d.DeptName
FROM STUDENT s
LEFT JOIN DEPARTMENT d ON s.DeptID = d.DeptID;
```

```
-- RIGHT JOIN
SELECT s.StudentName, d.DeptName
FROM STUDENT s
RIGHT JOIN DEPARTMENT d ON s.DeptID = d.DeptID;
```

```
-- CROSS JOIN
SELECT s.StudentName, d.DeptName
FROM STUDENT s
CROSS JOIN DEPARTMENT d;
```

9. Sample Output / Observations

Sr.	Step / Operation Performed	Output / Behaviour
1	INNER JOIN result	5 rows — only matched students and departments
2	LEFT JOIN result	5 rows — all students; NULL for missing dept
3	RIGHT JOIN result	6 rows — all 4 depts; Civil shows NULL student
4	CROSS JOIN result	$5 \times 4 = 20$ rows (Cartesian product)
5	INNER JOIN filter: Marks > 75	3 matched rows with marks above 75
6	LEFT JOIN where DeptName IS NULL	Students without a department shown
7	INNER JOIN row count	5 — same as STUDENT count (all have DeptID)

10. Result

INNER JOIN, LEFT JOIN, RIGHT JOIN, and CROSS JOIN were performed successfully. Each join type produced the expected result set, clearly demonstrating how different join types handle matched and unmatched rows.

11. Interpretation of Results

- INNER JOIN returned only 5 rows, confirming that all students had a matching department.
- RIGHT JOIN returned 6 rows, showing Civil Engineering with NULL student values — a department with no enrolled students.
- LEFT JOIN retained all students, with NULL appearing in DeptName for students without a department match.

- CROSS JOIN produced 20 rows (5×4), illustrating the Cartesian product and its potential for data explosion with large tables.

12. Precautions

- Always specify an ON condition for INNER/OUTER joins — omitting it performs a Cartesian product inadvertently.
- CROSS JOIN can produce very large result sets — use only when the Cartesian product is intentionally required.
- Use table aliases (s, d) to improve readability and resolve ambiguity when joining multiple tables.
- Verify that the join column data types match in both tables to avoid implicit conversion errors.

13. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Explain a scenario where LEFT JOIN is preferred over INNER JOIN.		✓	
2	Write a query to find all students along with their department names, including students with no department.			✓

Legend: R= Remember | U=Understand | A: Apply

Practical No: 11.1

1. Aim / Practical Statement

Exercise on various date and number format models

2. Practical Significance

Formatting date and number outputs is essential for generating professional, human-readable reports. SQL's format functions allow displaying data in locale-specific or application-specific formats directly within queries, reducing post-processing needs.

3. Relevant Theory

DATE_FORMAT() in MySQL

- DATE_FORMAT(date, format) returns a date value formatted according to the format string.
- Common format codes: %d (day), %m (month), %Y (4-digit year), %M (month name), %W (weekday name).
- Example: DATE_FORMAT(NOW(), '%d-%m-%Y') returns '24-04-2026'.

TO_CHAR() in Oracle

- TO_CHAR(date, 'format') is Oracle's equivalent for date formatting.
- Format elements: DD (day), MM (month), YYYY (year), MON (abbreviated month name).
- Example: TO_CHAR(SYSDATE, 'DD-MON-YYYY') returns '24-APR-2026'.

Number Format Models

- FORMAT(number, decimal_places) in MySQL formats a number with commas and decimal places.
- TO_CHAR(number, '999,999.99') in Oracle formats numbers with custom separators.
- Example: FORMAT(12345.678, 2) returns '12,345.68'.

LPAD / RPAD for Formatted Output

- LPAD(str, n, pad) pads the string on the left to length n using pad character.
- RPAD(str, n, pad) pads on the right.
- Useful for fixed-width report generation.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO4	3	3	2	3	–	1	2	2	3

Legend: 3 = High | 2 = Medium | 1 = Low | – = No Mapping

5. Lab Session Outcomes (LSOs)

After completing this practical, the student will be able to:

- Apply DATE_FORMAT to display dates in various formats.
- Use FORMAT to display numbers with commas and decimal precision.
- Apply TO_CHAR for date and number formatting in Oracle.
- Use LPAD and RPAD for fixed-width column formatting.
- Combine formatting functions with SELECT for report-style output.

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i3 or above, 4 GB RAM minimum
2	Operating System	Windows 10 / Ubuntu 20.04 or above
3	Database Software	MySQL 8.x / Oracle 21c XE
4	IDE / Editor	MySQL Workbench / Oracle SQL Developer

7. Suggested Procedure

1. Open the SQL editor and connect to CollegeDB.
2. Add a DOB (DATE) column to STUDENT and insert date values.
3. Apply DATE_FORMAT to display DOB in DD/MM/YYYY format.
4. Apply DATE_FORMAT to display the month name and weekday.
5. Use FORMAT to display Marks with 2 decimal places and commas.
6. Apply LPAD to display StudentID as a 6-character padded string.
7. Apply RPAD to right-pad StudentName to 20 characters.
8. Record all outputs in the observation table.
9. Write results in the lab journal.
10. Answer viva-voce questions.

8. Program Code / SQL Script

```
-- Add DOB column and insert data
ALTER TABLE STUDENT ADD DOB DATE;
UPDATE STUDENT SET DOB = '2005-06-15' WHERE StudentID = 101;
UPDATE STUDENT SET DOB = '2004-11-22' WHERE StudentID = 102;
UPDATE STUDENT SET DOB = '2005-03-08' WHERE StudentID = 103;
```

```
-- Date format models
SELECT StudentName,
       DATE_FORMAT(DOB, '%d/%m/%Y') AS FormattedDate1,
       DATE_FORMAT(DOB, '%D %M %Y') AS FormattedDate2,
       DATE_FORMAT(DOB, '%W, %d %M %Y') AS FullDateName
FROM STUDENT WHERE DOB IS NOT NULL;
```

```
-- Number format
SELECT StudentName, Marks,
       FORMAT(Marks, 2) AS FormattedMarks,
       LPAD(StudentID, 6, '0') AS PaddedID,
       RPAD(StudentName, 20, '.') AS PaddedName
FROM STUDENT;
```

```
-- Current date in various formats
SELECT DATE_FORMAT(NOW(), '%d-%m-%Y %H:%i:%s') AS DateTime1,
       DATE_FORMAT(NOW(), '%W, %d %M %Y')    AS DateTime2;
```

9. Sample Output / Observations

Sr.	Step / Operation Performed	Output / Behaviour
1	DATE_FORMAT(DOB, '%d/%m/%Y')	e.g., 15/06/2005
2	DATE_FORMAT with %D %M %Y	e.g., 15th June 2005
3	DATE_FORMAT with %W	e.g., Wednesday, 15 June 2005
4	FORMAT(Marks, 2)	e.g., 78.50 with 2 decimals
5	LPAD(StudentID, 6, '0')	e.g., 000101
6	RPAD(StudentName, 20, '.')	e.g., 'Amit Kumar.....'
7	FORMAT on large number (1234567.89)	1,234,567.89 with comma separation

10. Result

Date and number formatting functions were applied successfully. Dates were displayed in multiple formats using DATE_FORMAT, and numbers were formatted using FORMAT, LPAD, and RPAD for fixed-width, professional report-style output.

11. Interpretation of Results

- DATE_FORMAT with %D produced ordinal dates (15th), which is preferred in formal English reports.
- FORMAT applied comma separation to numeric values, making large numbers more readable.
- LPAD with '0' is commonly used to generate fixed-length codes such as employee or roll numbers.
- RPAD with '.' produced dot-padded names, simulating a tabular layout suitable for printed reports.

12. Precautions

- DATE_FORMAT is MySQL-specific; use TO_CHAR in Oracle for the same functionality.
- NULL date values return NULL in DATE_FORMAT — filter with IS NOT NULL for clean output.
- FORMAT returns a string, not a number — do not use the result in arithmetic operations.
- LPAD/RPAD do not modify stored data; they only affect the display in the query result.

13. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Write a query to display each student's name padded to 25 characters with their formatted DOB.			✓

Legend: R= Remember| U=Understand | A: Apply

Practical No: 12.1

1. Aim / Practical Statement

Exercise on creating tables with integrity constraints

2. Practical Significance

Integrity constraints ensure data accuracy and consistency in a database. They form the foundation of relational database integrity by enforcing business rules at the schema level, preventing invalid data from being stored.

3. Relevant Theory

PRIMARY KEY Constraint

- Uniquely identifies each record in a table; cannot be NULL or duplicate.
- A table can have only one PRIMARY KEY, which may be a composite key.
- Automatically creates a unique index on the column(s).

FOREIGN KEY Constraint

- Links a column in one table to the PRIMARY KEY of another table.
- Enforces referential integrity — prevents orphan records.
- Syntax: `col_name datatype REFERENCES parent_table(col);`

NOT NULL Constraint

- Ensures that a column cannot store NULL values.
- Syntax: `col_name datatype NOT NULL;`
- Applied during CREATE TABLE or ALTER TABLE.

UNIQUE and CHECK Constraints

- UNIQUE ensures all values in a column are distinct (allows one NULL).
- CHECK validates column values against a boolean expression.
- Syntax: `col_name datatype CHECK (condition);` e.g., `Age INT CHECK (Age >= 18).`

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO2	3	3	3	3	–	1	2	2	3

Legend: 3 = High | 2 = Medium | 1 = Low | – = No Mapping

5. Lab Session Outcomes (LSOs)

After completing this practical, the student will be able to:

- Create tables with PRIMARY KEY, NOT NULL, and UNIQUE constraints.

- Define a FOREIGN KEY constraint to establish table relationships.
- Apply CHECK constraints to validate column values.
- Verify constraint enforcement by attempting invalid data insertion.
- Drop and add constraints using ALTER TABLE.

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i3 or above, 4 GB RAM minimum
2	Operating System	Windows 10 / Ubuntu 20.04 or above
3	Database Software	MySQL 8.x / Oracle 21c XE
4	IDE / Editor	MySQL Workbench / Oracle SQL Developer

7. Suggested Procedure

1. Open the SQL editor and connect to CollegeDB.
2. Create a COURSE table with PRIMARY KEY and NOT NULL constraints.
3. Create an ENROLLMENT table with FOREIGN KEY referencing STUDENT and COURSE.
4. Apply UNIQUE constraint on Email and CHECK on Marks.
5. Insert valid records and verify acceptance.
6. Attempt to insert a duplicate PRIMARY KEY — observe error.
7. Attempt to insert a FOREIGN KEY value not in the parent table — observe error.
8. Attempt to violate CHECK constraint — observe error.
9. Record all outputs in the observation table.
10. Write results in the lab journal.
11. Answer viva-voce questions.

8. Program Code / SQL Script

```
-- Table with multiple constraints
CREATE TABLE COURSE (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(100) NOT NULL UNIQUE,
    Credits INT CHECK (Credits BETWEEN 1 AND 6)
);

INSERT INTO COURSE VALUES (1, 'Database Management', 4);
INSERT INTO COURSE VALUES (2, 'Java Programming', 4);

-- ENROLLMENT table with FOREIGN KEY
CREATE TABLE ENROLLMENT (
    EnrollID INT PRIMARY KEY,
    StudentID INT NOT NULL,
    CourseID INT NOT NULL,
    Grade CHAR(2),
    FOREIGN KEY (StudentID) REFERENCES STUDENT(StudentID),
    FOREIGN KEY (CourseID) REFERENCES COURSE(CourseID)
);
```

```

INSERT INTO ENROLLMENT VALUES (1, 101, 1, 'A');
INSERT INTO ENROLLMENT VALUES (2, 102, 2, 'B+');

-- Test constraint violations
-- PRIMARY KEY violation:
INSERT INTO COURSE VALUES (1, 'Data Structures', 3); -- ERROR

-- FOREIGN KEY violation:
INSERT INTO ENROLLMENT VALUES (3, 999, 1, 'C');    -- ERROR

-- CHECK violation:
INSERT INTO COURSE VALUES (3, 'AI Lab', 10);        -- ERROR

-- UNIQUE violation:
INSERT INTO COURSE VALUES (4, 'Database Management', 3); -- ERROR

```

9. Sample Output / Observations

Sr.	Step / Operation Performed	Output / Behaviour
1	Valid COURSE insert	2 rows inserted without errors
2	Valid ENROLLMENT insert	2 rows inserted without errors
3	Duplicate PRIMARY KEY insert	Error: Duplicate entry '1' for key PRIMARY
4	FOREIGN KEY violation (StudentID 999)	Error: Cannot add or update a child row
5	CHECK violation (Credits = 10)	Error: Check constraint violated
6	UNIQUE violation (duplicate CourseName)	Error: Duplicate entry for key UNIQUE
7	SELECT from ENROLLMENT with JOIN	Student names and course names correctly linked

10. Result

Tables were created with PRIMARY KEY, NOT NULL, UNIQUE, CHECK, and FOREIGN KEY constraints. All constraints were tested by attempting violations, and each produced the expected error message, confirming correct enforcement.

11. Interpretation of Results

- PRIMARY KEY enforcement prevented duplicate CourseID values, maintaining unique record identification.
- FOREIGN KEY constraint blocked orphan enrollment records, preserving referential integrity.
- CHECK constraint rejected Credits values outside the 1–6 range, enforcing the business rule.
- UNIQUE constraint blocked duplicate CourseName entries, ensuring course names remain distinct.

12. Precautions

- Insert data into parent tables before inserting into child tables that reference them via FOREIGN KEY.
- In MySQL, foreign key enforcement requires the InnoDB storage engine — MyISAM ignores foreign keys.
- CHECK constraints were not enforced in MySQL before version 8.0.16 — verify the version in use.
- Dropping a parent table or referenced column requires disabling foreign key checks first.

13. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Write a CREATE TABLE statement for EMPLOYEE with EmpID (PK), Name (NOT NULL), Salary (CHECK > 0), and DeptID (FK).			✓

Legend: R= Remember | U=Understand | A: Apply

Practical No: 13.1

1. Aim / Practical Statement

Exercise on creation and dropping of database and collections in MongoDB

2. Practical Significance

In MongoDB, databases and collections serve the same organizational role as schemas and tables in relational databases. Managing them effectively is the first step in NoSQL database administration and document-oriented application development.

3. Relevant Theory

MongoDB Databases

- A MongoDB database is a container for collections; one MongoDB instance can host multiple databases.
- use <dbName> switches to a database, creating it lazily on first data insertion.
- show dbs lists all databases that contain at least one document.

MongoDB Collections

- A collection is a group of MongoDB documents, analogous to a table in RDBMS.
- Collections are schema-less — different documents in the same collection can have different fields.
- Created explicitly with db.createCollection('name') or implicitly on first insert.

Dropping Databases and Collections

- db.dropDatabase() deletes the current database and all its collections permanently.
- db.<collection>.drop() removes a specific collection from the current database.
- Always confirm the current database using db before dropping.

MongoDB Shell Commands

- show dbs — list all non-empty databases.
- show collections — list all collections in the current database.
- use <name> — switch to or create a database.
- db — displays the name of the currently selected database.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO5	3	2	2	3	–	1	2	2	3

Legend: 3 = High | 2 = Medium | 1 = Low | – = No Mapping

5. Lab Session Outcomes (LSOs)

After completing this practical, the student will be able to:

- Create a new database in MongoDB using the use command.
- Create collections explicitly using db.createCollection().
- List available databases and collections using show commands.
- Drop a collection using db.<collection>.drop().
- Drop an entire database using db.dropDatabase().

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i3 or above, 4 GB RAM minimum
2	Operating System	Windows 10 / Ubuntu 20.04 or above
3	Database Software	MongoDB 6.x Community Edition
4	IDE / Tool	MongoDB Shell (mongosh) / MongoDB Compass

7. Suggested Procedure

1. Open MongoDB shell (mongosh) on the command prompt.
2. Type show dbs to list all existing databases.
3. Type use CollegeDB to create/switch to the CollegeDB database.
4. Type db to confirm the current database.
5. Execute db.createCollection('students') to create a collection.
6. Execute db.createCollection('courses') to create another collection.
7. Type show collections to verify creation.
8. Insert a document to make the database persistent: db.students.insertOne({name:'Test'}).
9. Execute db.courses.drop() to drop the courses collection.
10. Verify with show collections.
11. Execute db.dropDatabase() to drop CollegeDB.
12. Verify with show dbs and record all outputs.

8. Program Code / SQL Script

```
// Show existing databases  
show dbs
```

```
// Create / switch to database  
use CollegeDB
```

```
// Confirm current DB  
db
```

```
// Create collections explicitly  
db.createCollection('students')  
db.createCollection('courses')  
db.createCollection('faculty')
```

```
// List collections
```

```
show collections
```

```
// Insert to persist database  
db.students.insertOne({ name: "Test Student", branch: "CSE" })
```

```
// Verify database now appears in show dbs  
show dbs
```

```
// Drop a single collection  
db.courses.drop()
```

```
// Verify  
show collections
```

```
// Drop the entire database  
db.dropDatabase()
```

```
// Verify deletion  
show dbs
```

9. Sample Output / Observations

Sr.	Step / Operation Performed	Output / Behaviour
1	show dbs before CollegeDB creation	admin, config, local listed
2	use CollegeDB	switched to db CollegeDB
3	db.createCollection('students')	{ ok: 1 }
4	show collections	students, courses, faculty
5	insertOne into students	{ acknowledged: true, insertedId: ObjectId(...) }
6	db.courses.drop()	true
7	db.dropDatabase()	{ ok: 1, dropped: 'CollegeDB' }

10. Result

MongoDB databases and collections were created, verified, and dropped successfully. The show dbs and show collections commands confirmed all operations. The database persisted only after inserting a document, as expected in MongoDB's lazy creation model.

11. Interpretation of Results

- MongoDB does not persist a new database until at least one document is inserted, confirming its lazy creation model.
- db.createCollection() allows explicit collection creation before inserting any documents.
- db.courses.drop() returned true, confirming the collection was successfully deleted.
- db.dropDatabase() removed all collections and the database in a single operation.

12. Precautions

- Always verify the current database using `db` before executing `db.dropDatabase()` to avoid dropping the wrong database.
- MongoDB creates databases lazily — show dbs will not list a new database until it contains data.
- Dropping a collection or database is irreversible — ensure a backup exists before performing these operations.
- Collection names are case-sensitive in MongoDB — 'Students' and 'students' are different collections.

13. Viva-Voce Questions

S. N o.	Viva-Voce Question	R	U	A
1	Write the sequence of commands to create a database 'HospitalDB', create a collection 'patients', and verify it.			✓

Legend: R= Remember | U=Understand | A: Apply

Practical No: 15.1

1. Aim / Practical Statement

Exercise on MongoDB commands: Insert, update, find, delete, and sorting of documents

2. Practical Significance

CRUD operations (Create, Read, Update, Delete) are the core of any database interaction. In MongoDB, these operations work on documents using a flexible, JSON-based syntax that differs significantly from SQL, making this practical essential for NoSQL proficiency.

3. Relevant Theory

insertOne() and insertMany()

- `insertOne({})` inserts a single document into a collection.
- `insertMany([{}], {})` inserts multiple documents in one operation.
- MongoDB automatically adds an `_id` field (ObjectId) if not provided.

find() and findOne()

- `find({query})` retrieves documents matching the query; `{}` returns all documents.
- `findOne({query})` returns only the first matching document.
- Projections: `find({}, {field:1})` includes only specified fields (1=include, 0=exclude).

updateOne() and updateMany()

- `updateOne({filter}, {$set: {field: value}})` updates the first matching document.
- `updateMany()` updates all documents matching the filter.
- `$set` modifies specified fields; `$inc` increments numeric fields.

deleteOne() and deleteMany()

- `deleteOne({filter})` removes the first document matching the filter.
- `deleteMany({})` with empty filter deletes ALL documents — use with caution.
- Returns `{ acknowledged: true, deletedCount: n }` to confirm the operation.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2
CO5	3	2	2	3	–	1	2	2	3

Legend: 3 = High | 2 = Medium | 1 = Low | – = No Mapping

5. Lab Session Outcomes (LSOs)

After completing this practical, the student will be able to:

- Insert single and multiple documents using `insertOne()` and `insertMany()`.

- Query documents using find() with filter conditions and projections.
- Update documents using updateOne() and updateMany() with \$set operator.
- Delete documents using deleteOne() and deleteMany().
- Sort query results using the sort() method.

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i3 or above, 4 GB RAM minimum
2	Operating System	Windows 10 / Ubuntu 20.04 or above
3	Database Software	MongoDB 6.x Community Edition
4	IDE / Tool	MongoDB Shell (mongosh) / MongoDB Compass

7. Suggested Procedure

1. Open MongoDB shell and switch to CollegeDB: use CollegeDB.
2. Insert one document into students using insertOne().
3. Insert multiple documents using insertMany() with 4 student records.
4. Execute db.students.find() to retrieve all documents.
5. Execute find() with a filter: {branch: 'CSE'}.
6. Use projection: find({}, {name:1, marks:1, _id:0}).
7. Update one document: change marks for a specific student.
8. Update all CSE students: increment marks by 5 using \$inc.
9. Delete one student document by name.
10. Sort all students by marks descending using sort({marks:-1}).
11. Record all outputs in the observation table.
12. Write results in the lab journal.

8. Program Code / SQL Script

```
use CollegeDB
```

```
// Insert one document
```

```
db.students.insertOne({
  studentId: 101, name: "Amit Kumar",
  branch: "CSE", semester: 4, marks: 78.5
})
```

```
// Insert many documents
```

```
db.students.insertMany([
  { studentId: 102, name: "Priya Singh", branch: "CSE", semester: 4, marks: 85.0 },
  { studentId: 103, name: "Rahul Verma", branch: "ME", semester: 4, marks: 62.0 },
  { studentId: 104, name: "Neha Gupta", branch: "EE", semester: 4, marks: 90.5 },
  { studentId: 105, name: "Rohan Sharma", branch: "CSE", semester: 4, marks: 74.0 }
])
```

```
// Find all documents
```

```
db.students.find()
```

```

// Find with filter
db.students.find({ branch: "CSE" })

// Find with projection (include name & marks only)
db.students.find({}, { name: 1, marks: 1, _id: 0 })

// Update one document
db.students.updateOne(
  { studentId: 103 },
  { $set: { marks: 80.0 } }
)

// Update many: increment marks by 5 for CSE
db.students.updateMany(
  { branch: "CSE" },
  { $inc: { marks: 5 } }
)

// Delete one document
db.students.deleteOne({ name: "Rohan Sharma" })

// Sort by marks descending
db.students.find().sort({ marks: -1 })

// Sort by marks ascending
db.students.find().sort({ marks: 1 })

```

9. Sample Output / Observations

Sr.	Step / Operation Performed	Output / Behaviour
1	insertOne()	acknowledged: true, insertedId: ObjectId(...)
2	insertMany() — 4 documents	acknowledged: true, insertedIds: {0: ..., 3: ...}
3	find() — all documents	5 documents retrieved with all fields
4	find({ branch: 'CSE' })	3 documents for CSE branch
5	find() with projection	name and marks shown; _id excluded
6	updateOne — marks to 80.0	matchedCount: 1, modifiedCount: 1
7	sort({ marks: -1 })	Documents listed from highest to lowest marks

10. Result

All MongoDB CRUD operations were successfully executed. Documents were inserted using insertOne() and insertMany(), queried using find() with filters and projections, updated using updateOne() and updateMany(), deleted using deleteOne(), and sorted using sort().

11. Interpretation of Results

- `insertMany()` confirmed all 4 documents were acknowledged and assigned unique `ObjectId` values.
- `find({branch:'CSE'})` correctly filtered 3 out of 5 documents, demonstrating query filtering.
- `updateMany()` with `$inc` incremented marks for all 3 CSE students, confirming bulk update functionality.
- `sort({marks:-1})` returned documents in descending order of marks, useful for ranking and reporting.

12. Precautions

- Always use a specific filter with `deleteOne()` and `deleteMany()` — an empty filter `{}` deletes ALL documents.
- Use `$set` in `updateOne/updateMany` — without it, the entire document is replaced by the update value.
- MongoDB queries are case-sensitive — 'CSE' and 'cse' are treated as different values.
- The `find()` cursor in `mongosh` shows 20 results by default — type 'it' to iterate for more results.

13. Viva-Voce Questions

S. No.	Viva-Voce Question	R	U	A
1	Write a MongoDB command to update the semester from 4 to 5 for all CSE students.			✓
2	Write a <code>find()</code> command to retrieve the top 3 students sorted by marks in descending order.			✓

Legend: R= Remember | U=Understand | A: Apply